

Lesson 9: Session Management and Cookies

Marcin Kurzyna

Lecture Goals

This lecture introduces:

- What sessions are,
- What cookies are,
- How websites remember users,
- Session hijacking attacks,
- Secure cookie settings,
- Best practices for session management.

1 The Problem

HTTP is a stateless protocol. This means that every request is independent. For example:

- User logs in,
- User opens another page,
- Server receives a new request.

Without additional mechanisms, the server would not know who the user is.

2 What Is a Session?

A **session** is a way to remember a user between requests. After successful login:

- The server creates a session,
- The session receives a unique identifier,
- The identifier is sent to the browser.

The browser sends the identifier with future requests.

3 Session Identifier

A session identifier is usually a random string. Example:

```
A83F92D7BC11F2A9
```

The server stores information such as:

- Username,
- Login time,
- User permissions.

The browser stores only the session identifier.

4 What Is a Cookie?

A cookie is a small piece of data stored by the browser. Websites use cookies to:

- Store session identifiers,
- Remember preferences,
- Keep users logged in.

Example:

```
session_id=A83F92D7BC11F2A9
```

5 How Login Works

Simplified login process:

1. User enters username and password.
2. Server verifies credentials.
3. Server creates a session.
4. Session ID is stored in a cookie.
5. Browser sends the cookie with future requests.

6 Example in Flask

Creating a session:

```
from flask import Flask, session

app = Flask(__name__)
app.secret_key = "secret_key"

@app.route("/login")
```

```
def login():
    session["user"] = "alice"
    return "Logged in"
```

The session data is associated with the user.

7 Checking a Session

Example:

```
from flask import session

user = session.get("user")

if user:
    print("User is logged in")
else:
    print("User is not logged in")
```

8 Session Hijacking

Session hijacking occurs when an attacker steals a valid session identifier. If an attacker obtains:

```
session_id=A83F92D7BC11F2A9
```

the attacker may impersonate the victim.

9 How Sessions Are Stolen

Common methods include:

- XSS attacks,
- Malware,
- Network sniffing,
- Insecure HTTP connections.

This is one reason why HTTPS is important.

10 Secure Cookies

Browsers support additional cookie protections. **Secure**

- Cookie is sent only over HTTPS.

HttpOnly

- JavaScript cannot access the cookie.

SameSite

- Helps prevent Cross-Site Request Forgery attacks.

11 Setting Secure Cookies

Example:

```
app.config.update(  
    SESSION_COOKIE_SECURE=True,  
    SESSION_COOKIE_HTTPONLY=True,  
    SESSION_COOKIE_SAMESITE="Strict"  
)
```

These settings improve security.

12 Session Expiration

Sessions should not last forever. Reasons:

- Users forget to log out,
- Devices may be shared,
- Stolen sessions remain valid.

Applications should automatically expire inactive sessions.

13 Session Fixation

Session fixation occurs when an attacker forces a victim to use a known session ID. After login:

- The attacker already knows the session ID,
- The attacker may gain access to the account.

Applications should generate a new session after login.

14 Bad Security Practices

Examples:

- Predictable session IDs,
- Sessions that never expire,
- Storing passwords in cookies,
- Using HTTP instead of HTTPS,
- Missing HttpOnly protection.

These mistakes increase risk.

15 Secure Programming Recommendations

Developers should:

- Use HTTPS,
- Generate random session IDs,
- Enable Secure cookies,
- Enable HttpOnly cookies,
- Use session expiration,
- Create a new session after login.

Security should be enabled by default.

Summary

In this lecture, we covered:

- Sessions and cookies,
- Session identifiers,
- Login workflows,
- Session hijacking,
- Session fixation,
- Secure cookie settings,
- Best practices for session management.

16 Exercises

1. Explain why HTTP is called a stateless protocol.
2. What is the purpose of a session?
3. What information is usually stored in a cookie?
4. Explain the difference between:
 - Session data,
 - Session identifier.
5. What is session hijacking?
6. Why is the HttpOnly cookie attribute useful?

7. Why should sessions expire automatically?

8. Challenge:

Design a secure login system that:

- Uses sessions,
- Uses cookies,
- Protects against session hijacking,
- Uses HTTPS.